

Programare Orientată Pe Obiecte

Curs 8

Erori și Excepții

Noiembrie 2025

Introducere

- Oprim programul ?
- Returnam o valoare incorecta, setam un status ... dacă e posibil
- Verificăm ce a returnat functia → cod cu multe if-uri, greu de urmărit !

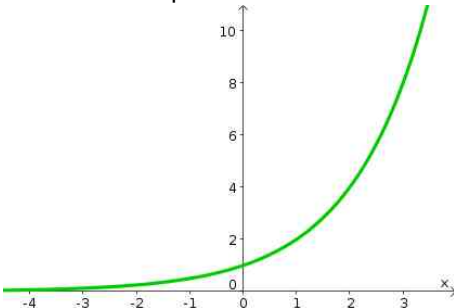
- De exemplu dacă se dorește citirea unui număr întreg cu `scanf("%d",&nr)` și de la tastatură se introduce un float ? Sau un char ?
- Ce returnează `scanf` ?
 - > 0 : The number of values converted and assigned successfully.
 - 0 : No value was assigned.
 - < 0 : Read error encountered or end-of-file(EOF) reached before any assignment was made.

Abordarea erorilor în Programarea Orientată pe Obiecte

- In Programarea Orientata pe Obiecte nu e nevoie sa folosim un cod de eroare;
- Daca metoda este executata cu succes atunci valoarea returnata e valida intotdeauna.
- **Exception:** pentru situatiile exceptionale care intrerup executia normala a unui program.
- **Exception:** un obiect care conține informație despre unde si de ce s-a oprit execuția normală a unei metode.
- Obiectul exceptie este aruncat *thrown* de o metodă și poate fi prins și tratat ulterior
- Execuția programului poate continua din punctul în care excepția a fost prinsă și tratată

De ce mecanisme de tratare a erorilor?

Costul de remediere a unei erori nerezolvate poate crește exponențial odată cu timpul:



Soluții:

- Mecanisme de tratare a excepțiilor
- Aserțiuni
- Testarea unitară

Excepții. Probleme în cursul execuției programelor

Un program întâlnește adesea probleme (excepții) în cursul execuției sale:

- poate avea probleme la citirea datelor
- pot exista caractere nepermise în date sau
- indexul unui tablou poate depăși limitele acestuia.

Excepțiile Java permit programatorului să trateze astfel de probleme:

- Putem scrie programe care își revin la întâlnirea excepțiilor și își continuă execuția
- Programele nu trebuie să eșueze atunci când utilizatorul face o greșeală!
- În special intrarea și ieșirea sunt susceptibile la excepții
- Tratarea excepțiilor este esențială pentru programarea I/E

Exemplu 1

Programul se rulează cu intrarea:

Enter one integer: h1

```
public class InputMismatchExceptionDemo {
    public static void main(String[] args) {
        Scanner kbd = new Scanner(System.in);
        System.out.print("Enter one integer:");
        int nb = kbd.nextInt();
        System.out.print("number is:" + nb);
        System.out.print("GATA CITIREA ! ");
    }
}
```

Programul nu este greșit dar are rezultatul:

```
java.util.InputMismatchException
    at java.util.Scanner.throwFor(Scanner.java:819)
    at java.util.Scanner.next(Scanner.java:1431)
    at java.util.Scanner.nextInt(Scanner.java:2040)
    at java.util.Scanner.nextInt(Scanner.java:2000)
    at InputMismatchExceptionDemo.main(InputMismatchExceptionDemo.java:11)
```

- Problema este că `nextInt` nu poate converti șirul de caractere "h1" la un `int`
- În momentul în care `nextInt` a întâlnit problema, metoda a aruncat o excepție de tipul `InputMismatchException`
- Sistemul de execuție Java a interceptat (a "prins") excepția, a oprit programul și a tipărit mesajele de eroare

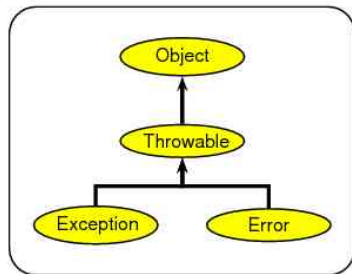
Excepții și erori

- O excepție: o problemă care apare în cursul execuției unui program.
 - La apariția unei excepții JVM creează un obiect de clasa `Exception` care conține informații despre problema apărută
 - Însuși programul Java poate **intercepta (catch)** o excepție. Apoi poate folosi obiectul de tipul excepție pentru a-și reveni după problemă.
- Și o eroare este o problemă care apare la rularea unui program
- O eroare este reprezentată de un obiect de clasa **Error**
- Dacă o eroare este prea severă pentru a fi tratată de un program programul trebuie să-și înceteze execuția.
- Exemplu: Out of memory error, System crash error

Ierarhia Throwable

Atât clasa **Exception** cât și clasa **Error** extind clasa **Throwable**.

- O metodă Java poate "arunca" un obiect de clasa **Throwable**
- D.e. `Integer.parseInt("zzz")` aruncă o excepție atunci când încearcă să convertească "zzz" într-un întreg



Excepții != Erori: se pot scrie programele astfel încât să-și revină după excepții, dar nu se pot scrie astfel încât să-și revină după erori.

Clasa Throwable

- Clasa Throwable contine (printre altele) metode pentru:
 - A descrie mesajul exceptiei
 - Stiva de urmarire (stack trace): secvență de apeluri de metodă înainte de apariția excepției
 - Exemplu: `main() → met1() → met2() → met3() → exceptie !`
 - Se poate localiza pozitia exacta a exceptiei (numele fisierului sursa, chiar si linia unde apare eroarea)
 - Se poate afisa stiva de apeluri
- Subclasele lui Throwable pot adauga informație suplimentară legată de Exceptie

Prinderea Excepțiilor

Exemplu 2:

Diferite tipuri de excepții

```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Introduceti un numar intreg: ");
    try {
        int val = scanner.nextInt();
        System.out.println("Nr introdus este " + val);
    } catch (InputMismatchException ex) {
        System.out.println("Exceptie prinsa: " + ex);
        ex.printStackTrace();
    }
    System.out.println("GATA CITIREA !");
}
```

La rulare:

```
Introduceti un numar intreg: abc
Exceptie prinsa: java.util.InputMismatchException
GATA CITIREA !
java.util.InputMismatchException
at java.base/java.util.Scanner.throwFor(Scanner.java:939)
at java.base/java.util.Scanner.next(Scanner.java:1594)
at java.base/java.util.Scanner.nextInt(Scanner.java:2258)
at java.base/java.util.Scanner.nextInt(Scanner.java:2212)
```

Exemplu 3:

Diferite tipuri de excepții

```
public static void main(String[] args) {  
    String[] sir = new String[]{ "10", "student", "5"};  
    for(int i=0; i<sir.length; i++) {  
        try {  
            int num = Integer.parseInt(sir[i]);  
            System.out.println(num);  
        } catch (NumberFormatException ex){  
            System.out.format("Excepție la pasul %d: %s%n", i, ex.getMessage());  
        }  
        System.out.println("Gata");  
    }  
}
```

La rulare se va afișa:

```
10  
Excepție la pasul 1: For input string: "student"  
5  
Gata
```

Exemplu 4:

Diferite tipuri de excepții

```
public static void main(String[] args) {
    String[] sir = new String[]{ "10", "student", "5"};
    for(int i=0; i<=sir.length; i++) {
        try {
            int num = Integer.parseInt(sir[i]);
            System.out.println(num);
        } catch (NumberFormatException ex){
            System.out.format("Excepție la pasul %d:%s%n",i,ex.getMessage());
        } }
        System.out.println("Gata");
    }
}
```

La rulare se va afișa:

```
10
Excepție la pasul 1: For input string: "student"
5
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
    Index 3 out of bounds for length 3 at Main.main(Main.java:20)
```

Exemplu 4:

Diferite tipuri de excepții

Multi-catch:

```
public static void main(String[] args) {
    String[] sir = new String[]{ "10", "student", "5"};
    for(int i=0; i<=sir.length; i++) {
        try {
            int num = Integer.parseInt(sir[i]);
            System.out.println(num);
        } catch (NumberFormatException ex){
            System.out.format("Excepție la pasul %d: %s\n",i,ex.getMessage());
        } catch (ArrayIndexOutOfBoundsException ex2){
            System.out.format("Excepție la pasul %d: %s\n",i,ex2.getMessage())
        }
    }
    System.out.println("Gata");
}
```

La rulare se va afișa:

```
10
Excepție la pasul 1: For input string: "student"
5
Excepție la pasul 3: Index 3 out of bounds for length 3
Gata
```

Exemplu 4:

Diferite tipuri de excepții

Multi-catch: dacă două blocuri catch folosesc același cod, se pot conecta cu operatorul |.

```
public static void main(String[] args) {
    String[] sir = new String[]{ "10", "student", "5"};
    for(int i=0; i<=sir.length; i++) {
        try {
            int num = Integer.parseInt(sir[i]);
            System.out.println(num);
        } catch (NumberFormatException | ArrayIndexOutOfBoundsException ex){
            System.out.format("Excepție la pasul %d: %s\n",i,ex.getMessage());
        } }
    System.out.println("Gata");
}
```

La rulare se va afișa:

```
10
Excepție la pasul 1: For input string: "student"
5
Excepție la pasul 3: Index 3 out of bounds for length 3
Gata
```

Tratarea Excepțiilor

Mecanismul try-throw-catch

- Calea fundamentală pentru tratarea excepțiilor în Java constă din trio-ul **try-throw-catch**
- Blocul try conține codul pentru algoritmul implementat
- Acest cod spune ce se face atunci când totul merge bine
- Se numește bloc try deoarece el "încearcă" să execute cazul în care totul merge așa cum a fost planificat
- De asemenea acest bloc poate conține cod care aruncă o excepție dacă se întâmplă ceva neobișnuit:

```
try {  
    CodCarePoateAruncaOExcepție  
}
```

Ce se întâmplă la executarea unui bloc **try**?

Se pot întâmpla două lucruri:

- 1 Nu este aruncată nici o excepție în blocul try
 - Codul din blocul try este executat până la sfârșitul blocului
 - Blocul catch este sărit
 - Execuția continuă de la codul amplasat după blocul catch
- 2 Este aruncată o excepție în blocul try și interceptată în blocul catch
 - Restul codului din blocul try este sărit
 - Controlul se transferă la un bloc catch următor (în cazurile simple)
 - Obiectul aruncat este transmis ca parametru al blocului catch
 - Se execută codul din blocul catch
 - Se execută codul care urmează după blocul catch respectiv (dacă există)

Aruncarea explicită a unei excepții

- La aruncarea unei excepții, execuția blocului **try** în care a fost aruncată excepția se oprește
- Normal, controlul este transferat unei alte porțiuni de cod, blocul **catch** (blocul de interceptare)
- Valoarea aruncată este argumentul operatorului throw; ea este întotdeauna un obiect aparținând unei clase excepție
- Execuția unei instrucțiuni **throw** se numește aruncare a unei excepții
- O instrucțiune **throw** seamănă cu un apel de metodă

```
throw new NumeleClaseiExceptie(PosibilArgumente);
```

- În exemplul de mai sus, obiectul de clasa `NumeClasaExceptie` este creat folosind ca argument un șir de caractere
- Acest obiect, care este argument pentru operatorul `throw`, este obiectul excepție aruncat
- În loc să apeleze o metodă, instrucțiunea `throw` apelează un bloc `catch`

Exemplu 5:

Exemplu cu throw

```
public class Main {  
    public static int perimetruDreptunghi(int a, int b, int c, int d){  
        return a + b + c + d;  
    }  
  
    public static void main(String[] args) {  
        int p = perimetruDreptunghi (2,3,4,5);  
    }  
}
```

- Ce e in neregula cu codul ? Sau cu logica programului ?
- Daca laturile a, b,c,d nu formeaza un dreptunghi ?

Exemplu 5:

Exemplu cu throw

```
public class Main {
    public static int perimetruDreptunghi(int a, int b, int c, int d){
        if (!esteDreptunghi(a,b,c,d)) throw new IllegalArgumentException(
            String.format("(%d,%d,%d,%d) nu e dreptunghi", a, b, c, d));
        return a + b + c + d;
    }
    public static void main(String[] args) {
        int p = perimetruDreptunghi (2,3,4,5);
    }
    static boolean esteDreptunghi(int a, int b, int c, int d) {
        if (a == d and b == c and b == d) return true;
        else if (a == b and c == d) return true;
        else if (a == d and c == b) return true;
        else if (a == c and d == b) return true;
        else return false;
    }
}}
```

La rulare se va afisa:

```
Exception in thread "main" java.lang.IllegalArgumentException:
    (2,3,4,5) nu e dreptunghi
at Main.perimetruDreptunghi(Main.java:23)
at Main.main(Main.java:27)
```

Mecanismul try-throw-catch

- La aruncarea unei excepții se începe executarea blocului catch
 - Blocul catch are un parametru
 - Obiectul excepție aruncat este transmis ca parametru al blocului catch
- Un bloc catch este o porțiune de cod separată care se execută atunci când un program întâlnește și execută o instrucțiune throw în blocul try precedent
- Execuția blocului catch se numește interceptarea/"prinderea" excepției, sau tratarea excepției

```
catch (Exception e) { CodDeTratareAExceptiei }
```

- Identificatorul **e** din blocul **catch** de deasupra se numește **parametru al blocului catch** și îndeplinește două roluri:
 - ➊ Specifică tipul de obiect excepție aruncat pe care blocul catch îl poate intercepta (d.e., mai sus este un obiect de clasa Exception)
 - ➋ Oferă un nume (pentru obiectul care este interceptat) care să fie folosit în blocul catch
 - ➌ Observație: adesea se folosește identificatorul **e** prin convenție, dar se poate folosi orice identificator care nu este cuvânt cheie.

Blocuri catch multiple

- Un bloc try poate arunca potențial orice număr de valori excepție, iar acestea pot fi de tipuri diferite
 - În oricare execuție a unui bloc try, poate fi aruncată cel mult o excepție (de vreme ce instrucțiunea throw termină execuția blocului try)
 - La execuții diferite ale blocului try pot fi aruncate valori diferite
- Fiecare bloc catch poate intercepta valorile de tipul de clasă excepție, date în antetul blocului catch
- Se pot intercepta tipuri diferite de excepții punând mai multe blocuri catch după un bloc try
 - Se pot pune oricâte blocuri catch, dar în ordinea corectă

Blocuri catch multiple

Exemplu

```
public class TestMultipleCatchBlock {  
    public static void main(String args[]){  
        try{  
            int a[]=new int[5];  
            a[5]=30/0;  
        } catch(ArithmeticException e){System.out.print("task1 done");  
        } catch(ArrayIndexOutOfBoundsException e){  
            System.out.println("task2 done");  
        } catch(Exception e){System.out.println("common task completed");}  
        System.out.println("rest of the code...");  
    } }
```

Se va afisa:

```
task1 done  
rest of the code...
```

Observați că se execută mai întâi 30/0 și se generează
ArithmeticException !

Blocuri catch multiple

Exemplu

```
public class TestMultipleCatchBlock {
    public static void main(String args[]){
        try{
            int a[]=new int[5];
            a[5]=30/2;
        } catch(ArithmeticException e){System.out.print("task1 done");
        } catch(ArrayIndexOutOfBoundsException e){
            System.out.println("task2 done");
        } catch(Exception e){System.out.println("common task completed");}
        System.out.println("rest of the code...");
    }
}
```

Se va afisa:

```
task2 done
rest of the code...
```

Pentru că avem $a[5] = 30/2$ iar șirul `a` are ultimul element pe `a[4]` !

- La aruncarea unei excepții într-un bloc try, blocurile catch sunt examinate în ordinea apariției
- Este executat primul bloc care se potrivește cu tipul de excepție aruncat

- Deoarece **NegativeNumberException** este un tip de **Exception**, toate NegativeNumberExceptions vor fi interceptate de către primul bloc catch înainte de a ajunge vreodată la cel de-al doilea
- Blocul catch pentru NegativeNumberException nu va fi folosit!
- Pentru ordine corectă, inversați cele două blocuri

Interceptați mai întâi cea mai specifică excepție

Exemplu

```
public class DublaGreseala {
    public static void main(String[] args) {
        int num = 5, denom = 0, result;
        int[] arr = {7, 21, 31};
        try {
            result = num / denom;
            result = arr[num];
        } catch (ArithmeticException ex) {
            System.out.println("Eroare aritmetica");
        } catch (IndexOutOfBoundsException ex) {
            System.out.println("Eroare de indice");
        }
    }
}
```

Observație: Cea de a doua excepție nu va fi aruncată niciodată!

Logica mecanismului try-throw-catch

- La aruncarea unei excepții de către o instrucțiune blocul try, blocurile catch sunt examinate unul câte unul începând cu primul
- Un singur bloc catch este ales
- Dacă nici un bloc catch nu se potrivește cu excepția, atunci nu este ales nici unul, iar execuția părăsește metoda respectivă (exact ca în lipsa blocului catch)
- Primul bloc catch care se potrivește cu tipul de excepție aruncată obține controlul
- Cele mai specifice tipuri de excepție trebuie să apară la început, urmate de tipurile mai generale de excepție
- Instrucțiunile din blocul catch ales sunt executate secvențial; după executarea ultimei instrucțiuni, controlul ajunge la prima instrucțiune care urmează după structura try/catch
- Controlul nu se întoarce în blocul try

Clauza finally

Clauza finally

- Excepția provoacă terminarea metodei curente
- Pericol: se poate sări peste o porțiune de cod esențială

```
reader = new FileReader(filename);  
Scanner in = new Scanner(reader);  
readData(in);  
reader.close(); // s-ar putea sa nu ajunga aici niciodata
```

- Trebuie executat reader.close() chiar dacă apare o excepție
- **Folosim clauza finally pentru codul care trebuie executat "indiferent de ce se întâmplă" (necondiționat)**

Blocuri catch multiple și clauza finally

Dacă există clauze catch asociate blocului try, atunci trebuie să punem clauza finally după toate clauzele catch.

```
try {
// Bloc de cod cu puncte de iesire multiple
} catch (OneException e) {
    System.out.println(" Am interceptat OneException!");
} catch (OtherException e) {
    System.out.println(" Am interceptat OtherException!");
} catch (AnotherException e) {
    System.out.println(" Am interceptat AnotherException!");
} finally {
    // Bloc de cod executat intotdeauna la iesirea din bloc,
    // indiferent de cum s-a iesit din "try"
    System.out.println("Finally este executat intotdeauna");
}
```

Clauza finally exemplu

Ce se va afișa la rularea următorului cod:

```
public static void main(String[] args){
String[] sir = new String[]{ "10", "student", "5"};
for(int i=0; i<=sir.length; i++) {
    try {
        int num = Integer.parseInt(sir[i]);
        System.out.println(num);
    }catch (NumberFormatException | ArrayIndexOutOfBoundsException ex){
        System.out.format("Exceptie pasul %d: %s\n", i, ex.getMessage());
    } finally {    System.out.print("Finally se executa !"); } }
System.out.println("Gata"); }
```

```
10
Finally se executa !
Exceptie pasul 1: For input string
: "student"
Finally se executa !
5
Finally se executa !
Exceptie pasul 3: Index 3 out of
    bounds for length 3
Finally se executa !
Gata
```

- Nu e nevoie să apară o excepție pentru ca să se execute blocul finally.
- Blocul finally se execută întotdeauna.
- Finally nu se execută dacă apare o eroare care cauzează System.exit (out of memory) RAR !

try și finally fără catch

```
public class DemoTryFinally {  
    public static void main(String[] args) {  
        try {  
            System.out.println("Try Block");  
        } finally {  
            System.out.println("Finally Block");  
        } } }
```

Se afiseaza:

Try Block

Finally Block

try și finally fără catch

Explicații

- **Pasul 1: Execuția începe în blocul try:**

- Instrucțiunea `System.out.println("Try Block cu return");` este executată.
- Se afișează: Try Block cu return.
- Instrucțiunea `return 10;` "pregătește" returnarea valorii 10.

- **Pasul 2: Blocul finally se execută:**

- Blocul finally se execută întotdeauna, indiferent dacă în try există un return.
- Se execută instrucțiunea `System.out.println("Finally Block se executa");`.
- Se afișează: Finally Block se executa.

- **Pasul 3: Returnarea valorii**

- După executarea blocului finally, metoda revine la instrucțiunea return.
- Valoarea 10, pregătită anterior, este returnată către apelant (main).
- În main, `System.out.println(demo());` afișează valoarea 10.

Definirea excepțiilor

Clase Exceptie

Există mai multe clase excepție pe lângă clasa Exception:

- Există mai multe clase excepție în bibliotecile standard Java
- Pot fi definite noi clase excepție exact ca orice alte clase

Toate clasele excepție predefinite au următoarele proprietăți:

- Posedă un constructor cu un singur argument de tipul String
- Clasa are o metoda accesoare, getMessage(), care poate recupera șirul dat ca argument constructorului la crearea obiectului excepție

Toate clasele excepție definite de programator ar trebui să aibă aceleași proprietăți

Clase Excepție din Pachetele Standard

Există numeroase clase excepție predefinite care sunt incluse în pachetele standard Java:

- IOException
- NoSuchMethodException
- FileNotFoundException
- Multe clase excepție trebuie importate pentru a le putea utiliza:

```
import java.io.IOException;
```

Clasa predefinită Exception este clasa rădăcină pentru toate excepțiile:

- Fiecare clasă excepție este descendentă din clasa Exception
- Clasa Exception poate fi folosită: direct sau pentru a defini o clasă derivată
- Clasa Exception se află în pachetul java.lang și nu trebuie clauză import

Folosirea metodei `getMessage`

```
... // codul metodei
try {
    ...
    throw new Exception(StringArgument)
} catch (Exception e) {
    String message = e.getMessage();
    System.out.print(message);
    System.exit(0);
} ...
```

Fiecare excepție are o variabilă instanță de tipul `String` care conține un mesaj:

- Acest șir identifică de obicei motivul apariției excepției

`StringArgument` este folosit ca valoare pentru variabila instanță de tip șir a excepției **`e`**:

- De aceea, apelul de metodă **`e.getMessage()`** returnează acest șir

Definire clase exceptie

- Fiecare clasă excepție care urmează să fie definită trebuie să fie o clasă derivată dintr-o clasă excepție deja definită
 - Derivată din oricare clasă excepție definită în bibliotecile standard Java sau definită de către programator
- Constructorii sunt membrii cei mai importanți în definirea unei clase excepție
 - Constructorii trebuie să se comporte corespunzător în raport cu variabilele și metodele moștenite din clasa de bază
 - Adesea, nu există alți membri cu excepția celor moșteniți din clasa de bază
- Clasa care urmează nu efectuează decât aceste sarcini fundamentale

Clasă excepție definită de programator

```
public class DivisionByZeroException extends Exception
{
    public DivisionByZeroException() {
        /* Se poate face mai mult intr-un constructor de excepție, dar
           aceasta este o forma uzuala */
        super("Division by zero.");
    }
    public DivisionByZeroException(String message) {
        super(message);
        /* super invoca constructorul clasei de baza Exception */
    } }
}
```

Caracteristicile obiectului Exception

Cele mai importante două lucruri referitoare la un obiect excepție sunt tipul său (adică, clasa excepție) și mesajul pe care îl poartă:

- Mesajul este transmis împreună cu obiectul excepție ca variabilă instanță
- Acest mesaj poate fi recuperat cu metoda accesoare getMessage, astfel că blocul catch poate folosi mesajul

Indicații pentru clasele excepție definite de programator

- Clasele excepție pot fi definite de către programator, dar fiecare asemenea clasă trebuie să fie derivată dintr-o clasă excepție existentă deja
- Clasa Exception poate fi folosită pe post de clasă de bază, cu excepția cazului în care o altă clasă excepție este mai potrivită
- Trebuie definiți cel puțin doi constructori, iar uneori mai mulți
- Excepția trebuie să țină seama că metoda getMessage() este moștenită

Indicații pentru clasele excepție definite de programator

Păstrează `getMessage ()`

- Pentru toate clasele excepție predefinite, `getMessage` returnează șirul de caractere transmis ca argument constructorului său
 - Sau trebuie să returneze un șir implicit dacă nu s-a transmis nici un argument constructorului
- Acest comportament trebuie păstrat în toate clasele excepție definite de către programator:

Pași pentru definirea unei excepții personalizate

- 1 Creează o clasă care extinde `Exception` (pentru o excepție verificată) sau `RuntimeException` (pentru o excepție neverificată).
- 2 Definește un constructor fără argumente (opțional).
- 3 Definește un constructor care acceptă un mesaj (`String`) pentru a descrie eroarea.
- 4 Poți adăuga alte metode și câmpuri dacă sunt necesare.

Exemplu: Definirea unei excepții personalizate

```
class CustomException extends Exception {  
    // Constructor fara argumente  
    public CustomException() {  
        super("Exceptie personalizata generata");  
    }  
  
    // Constructor cu mesaj personalizat  
    public CustomException(String message) {  
        super(message);  
    }  
}
```


Explicație: Codul anterior

- `CustomException` este o clasă de excepție verificată (`extends Exception`).
- Metoda `validateAge()` verifică dacă vârsta este sub 18 ani.
- Dacă regula este încălcată, metoda aruncă excepția personalizată folosind `throw`.
- În metoda `main()`, excepția este gestionată într-un bloc `try-catch`.

Aruncarea excepțiilor

Regula: "prinde sau declară"

Cele mai obișnuite excepții care ar putea fi aruncate într-o metodă trebuie tratate în unul dintre următoarele două moduri:

- 1 Codul care poate arunca o excepție este pus într-un bloc **try**, iar excepția care poate apărea este interceptată într-un bloc **catch** din **aceeași metodă**
- 2 Excepția posibilă poate fi declarată la începutul definiției metodei punând numele clasei excepție într-o clauză **throws**
 - Reprezintă o modalitate de a deplasa răspunderea pentru tratarea excepției la metoda care a invocat-o pe cea care a aruncat excepția
 - Metoda apelantă trebuie să trateze excepția, cu excepția cazului în care folosește aceeași tehnică de "pasare"
 - Într-un sfârșit, fiecare excepție ar trebui interceptată de un bloc catch din vreo metodă care nu numai declară într-o clauză throws ci și interceptează clasa de excepție respectivă

Regula: "prinde sau declară"

- Cele două tehnici pot fi amestecate: Unele excepții pot fi interceptate, iar altele declarate în clauza throws.
- Cu toate acestea, tehnicile menționate trebuie folosite consistent pentru o excepție dată
 - Dacă o excepție nu este declarată, atunci ea trebuie tratată în metodă
 - Dacă este declarată excepția, atunci responsabilitatea pentru tratarea ei este pasată unei alte metode care o apelează
 - Observați că dacă definiția unei metode include invocarea unei a doua metode, iar cea de a doua poate arunca o excepție și nu o interceptează, atunci prima metodă trebuie să o declare sau să o intercepteze

Aruncarea excepțiilor într-o metodă

- Aruncarea unei excepții implică utilizarea cuvântului cheie `throw`.
 - Folosit pentru a genera o excepție explicit.
 - Se utilizează împreună cu un obiect care extinde clasa `Throwable`.

```
throw new ExceptionType("Mesaj de eroare");
```

- O metodă care poate arunca o excepție verificată trebuie să declare acest lucru folosind `throws`.
 - Declară că o metodă poate arunca una sau mai multe excepții.

```
public void methodName() throws ExceptionType {  
    // Codul metodei  
}
```

- **Este obligatoriu pentru excepțiile verificate.**

Clauza `throws` în Java

- **throws** se folosește în declarația unei metode.
- Indică faptul că metoda poate arunca una sau mai multe excepții.
- Transferă responsabilitatea tratării excepției către:
 - metoda apelantă (care va folosi `try/catch`), sau
 - o metodă superioară care continuă propagarea.
- Este obligatoriu pentru **excepțiile verificate** (checked).
- Nu este necesar pentru **excepțiile neverificate** (unchecked).
- Folosit când metoda nu poate decide singură ce să facă în caz de eroare.

Exemplu cu throws

```
class InvalidNumberException extends Exception {
    public InvalidNumberException(String message) {
        super(message);
    }
}

public class ExempluThrows2 {
    // Metoda care poate arunca o exceptie - declaram throws
    static int convertToInt(String text) throws InvalidNumberException {
        try { return Integer.parseInt(text);
        } catch (NumberFormatException e) {
            throw new InvalidNumberException(text+" nu este un numar valid!");
        }
    }

    public static void main(String[] args) {
        try {
            int nr = convertToInt("123");
            System.out.println("Numar valid: " + nr);
            nr = convertToInt("abc");    // va arunca exceptie
            System.out.println("Numar valid: " + nr);
        } catch (InvalidNumberException e) {
            System.out.println("Eroare: " + e.getMessage());
        }
    }
}
```

Clauza throws în clase derivate

Ce se întâmplă la suprascrierea unei metode (care aruncă o excepție)?

- 1 Clauza **throws** a metodei din clasa copil/derivată poate declara **aceleași excepții ca metoda din clasa de bază**
- 2 Clauza **throws** a metodei din clasa copil/derivată poate **reduce numărul sau tipul excepțiilor aruncate**
- 3 Clauza **throws** din clasa derivată **nu poate adăuga excepții** mai generale sau noi (checked exceptions)

Clauza `throws` a metodei din clasa derivată declară aceleași excepții ca metoda din clasa de bază

```
class BaseClass {  
    public void method() throws IOException {  
        // Cod care poate arunca IOException  
    }  
}  
  
class DerivedClass extends BaseClass {  
    @Override  
    public void method() throws IOException {  
        // Cod care poate arunca IOException  
    }  
}
```

Clauza throws din clasa derivată poate reduce numărul sau tipul excepțiilor aruncate

- Este permis să se elimine complet clauza throws dacă metoda din clasa derivată nu aruncă excepții.

```
class BaseClass {
    public void method() throws IOException {
        // Cod care poate arunca IOException
    } }
class DerivedClass extends BaseClass {
    @Override
    public void method() {
        // Nu mai arunca nicio exceptie
    } }
```

Clauza throws din clasa derivată nu poate adăuga excepții mai generale sau noi (checked exceptions):

- Metoda suprascrisă nu poate declara o excepție mai generală decât cea din clasa de bază.
- Se poate declara o excepție mai specifică (subclasă a celei declarate în clasa de bază).
- De asemenea, nu poate adăuga excepții checked care nu sunt prezente în clasa de bază.

```
class Baza {
    public void method() throws IOException {
        // Cod care poate arunca IOException
    }
}

class DerivataNOTOK extends Baza {
    @Override
    public void method() throws Exception { // Eroare! Exception este
        mai generala decat IOException
    }
}

class DerivataOK extends Baza {
    // OK: arunca o exceptie mai "ingusta"
    @Override
    void metoda() throws FileNotFoundException { }
```

Categorii de excepții în JAVA

În Java există două categorii principale de excepții:

- **Excepții verificate (checked):** Excepțiile care sunt supuse regulii "prinde sau declară"
- **Excepții neverificate (unchecked)**

Acestea diferă prin modul în care sunt tratate și impuse de compilator.

Excepții verificate (checked)

- Compilatorul **obligă** programatorul să le trateze.
- Trebuie folosit `try/catch` sau `throws`.
- Moștenesc `Exception`, dar **nu** `RuntimeException`.
- Apar în situații **previzibile** și **recuperabile**.
- **Exemple:**
 - `IOException`
 - `SQLException`
 - `ClassNotFoundException`
 - `ParseException`

Exemplu: Gestionarea unei excepții verificate

```
import java.io.*;

public class CheckedExample {
    public static void main(String[] args) {
        try {
            readFile("path_to_file.txt");
        } catch (IOException e) {
            System.out.println("Excepție gestionată: " + e.getMessage());
        }
    }

    public static void readFile(String filePath) throws IOException {
        FileReader file = new FileReader(filePath);
        BufferedReader br = new BufferedReader(file);
        System.out.println(br.readLine());
        br.close();
    }
}
```

Excepții neverificate (unchecked)

- Compilatorul **nu obligă** tratarea lor.
- Moștenesc `RuntimeException`.
- Reprezintă **erori de programare** (bug-uri).
- De obicei NU se tratează cu `try/catch`.
- **Exemple:**
 - `NullPointerException`
 - `ArithmeticException`
 - `ArrayIndexOutOfBoundsException`
 - `IllegalArgumentException`
 - `NumberFormatException`

Exemplu: Gestionarea unei excepții neverificate

```
public class UncheckedExample {  
    public static void main(String[] args) {  
        int[] numbers = {1, 2, 3};  
        try {  
            System.out.println(numbers[5]); // Acces invalid  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Excepție gestionată: " + e.getMessage());  
        }  
        try {  
            int result = 10 / 0; // Impartire la zero  
        } catch (ArithmeticException e) {  
            System.out.println("Excepție gestionată: " + e.getMessage());  
        }  
    }  
}
```

```
import java.io.*;

public class CheckedExceptionExample {
    public static void main(String[] args) {
        try {
            readFile("path_to_file.txt");
        } catch (IOException e) {
            System.out.println("Exceptie gestionata: " + e.getMessage());
        } }

    // Declararea exceptiei folosind 'throws'
    public static void readFile(String filePath) throws IOException {
        FileReader file = new FileReader(filePath); // Poate arunca
            FileNotFoundException
        BufferedReader br = new BufferedReader(file);
        System.out.println(br.readLine());
        br.close(); } }
```

- `readFile()` declară că poate arunca o excepție de tip `IOException`
- Excepția e generată dacă fișierul specificat nu există sau nu poate fi citit
- În `main()`, metoda este apelată în blocul `try-catch`, unde excepția este gestionată

Exemplu: Aruncarea unei excepții neverificate

```
public class UncheckedExceptionExample {
    public static void main(String[] args) {
        try {
            validateNumber(-5); // Va genera exceptia
        } catch (IllegalArgumentException e) {
            System.out.println("Exceptie gestionata: " + e.getMessage());
        }
    }
    // Nu este necesara declaratia 'throws' pentru exceptiile neverificate
    public static void validateNumber(int number) {
        if (number < 0) {
            throw new IllegalArgumentException("Nr nu poate fi negativ!");
        }
    }
}
```

-
- ```

graph TD
 A[Method with exception] -- "Throws exception" --> B[Method without an exception handler]
 B -- "Forwards the exception to previous method" --> C[Method without suitable handler]
 C -- "Handler doesn't match
Forwards exception" --> D[Method with suitable handler]
 D -- "Catches the exception" --> E[main]

```

lu 65 / 70

# Re-aruncarea excepțiilor

- După interceptarea unei excepții, ea poate fi re-aruncată dacă e cazul
- La re-aruncarea unei excepții putem alege locația din care se va vedea ca aruncat obiectul în trasarea stivei de execuție
  - Putem face ca excepția re-aruncată să pară a fi aruncată din locul excepției originale sau din locul re-aruncării
- Pentru a re-arunca o excepție cu indicarea locației originale, pur și simplu o aruncăm din nou

```
try {
 cap(0);
} catch (ArithmeticException e) { throw e; }
```

- ```
try {
    cap(0);
} catch (ArithmeticException e) {
    throw (ArithmeticException)e.fillInStackTrace();
}
```

- 14 67 / 70

Exemplu cu fillInStackTrace()

```
public class ThrowableDemo {
    public static void function1() throws Exception {
        throw new Exception("Sursa:function1()"); }
    public static void function2() throws Throwable{
        try { function1(); }
        catch(Exception e) {
            System.err.println("Inside function2():");
            e.printStackTrace();
            throw e.fillInStackTrace();
        }}
    public static void main(String[] args) throws Throwable {
        try { function2(); }
        catch (Exception e) {
            System.err.println("Caught Inside Main:");
            e.printStackTrace();
        } } }
}
```

Se va afisa:

```
Inside function2():
java.lang.Exception: Sursa:function1() at Main.function1(Main.java:29)
at Main.function2(Main.java:32) at Main.main(Main.java:41)
Caught Inside Main: java.lang.Exception: Sursa:function1()
at Main.function2(Main.java:36) at Main.main(Main.java:41)
```

Aserțiuni în Java

- Aserțiunile sunt niște mecanisme ce permit testarea logicii programului în faza de dezvoltare. Folosirea lor e limitată și specifică anumitor cazuri.
- Aserțiunile sunt verificări interne folosite în faza de dezvoltare.
- Se scriu cu cuvântul cheie `assert`.
- Nu sunt destinate tratării erorilor de la utilizator, ci depistării bug-urilor.
- Sunt **dezactivate** implicit și se activează cu `-ea`.
- În faza de producție aserțiunile se dezactivează, programul rulând normal fără a ține cont de existența aserțiunilor
- Format:
 - `assert condiție;`
 - `assert condiție : mesajEroare;`

Exemplu practic cu aserțiuni

```
static int impartirePozitiva(int a, int b) {
    assert a >= 0 : "a este negativ!";
    assert b > 0 : "b trebuie sa fie > 0";
    return a / b;
}

public static void main(String[] args) {
    int r = impartirePozitiva(10, 2);
    System.out.println(r);

    // Activeaza asertiunile:
    // java -ea ExempluAserțiuni
    r = impartirePozitiva(-5, 0);
}
```